

Boosting and reading a model

Machine Learning in Finance · Session 12

The models so far

	model	defined by	test score
a number	linear regression	$\min_{\beta} \sum_i (y_i - \beta_0 - \sum_j \beta_j x_{ij})^2$	RMSE 0.237 on <code>vol_20d</code>
	ridge	$\min_{\beta} \text{RSS} + \alpha \sum_j \beta_j^2$	RMSE 0.218
	lasso	$\min_{\beta} \text{RSS} + \alpha \sum_j \beta_j $	RMSE 0.217
a label	logistic regression	$P(y = 1) = \sigma(\beta_0 + \sum_j \beta_j x_j)$	AUC 0.748
	k-nearest neighbours	$\hat{p} = \frac{1}{k} \sum_{k \text{ nearest}} y_i$	AUC 0.749
both	a tree	$\hat{f}(\mathbf{x}) = \sum_m c_m \mathbf{1}\{\mathbf{x} \in R_m\}$	RMSE 0.241, AUC 0.722
	a random forest	$\hat{f}(\mathbf{x}) = \frac{1}{B} \sum_b \hat{f}^b(\mathbf{x})$	RMSE 0.226, AUC 0.777

A forest grows deep trees side by side and averages them. Today's model grows small trees one after another, each fitted to what the ones before still get wrong.

The volatility table

Python Code

```
from sklearn.metrics import mean_squared_error

table = pd.read_csv("data/market_features.csv", parse_dates=["date"]).set_index("date")
train, test = table.loc[:"2022-12-31"], table.loc["2023-01-01":]
columns = list(table.columns[:-1])
print(len(train), len(test), len(columns))

def rmse(actual, predicted):
    return np.sqrt(mean_squared_error(actual, predicted))
```

1894 482 19

The table and split that ridge and the forest were scored on: one row per trading day, 19 columns, and **vol_next**, the index's volatility over the next 20 days.

Today

- what **averaging** cannot fix, and **adding** trees instead
- **boosting**: the definition, the procedure, and the procedure by hand
- how many **trees**, and stopping **early**
- **XGBoost** and **LightGBM**, the libraries used in practice
- boosting a **label**, on the credit table
- **reading** a model: which columns it uses
- the course's models, **side by side**

The exercises and the case are where you practise each of these.

1 • What averaging cannot fix

PART 1

A forest removes the variance of deep trees. Small trees make a different error.

One stump, and 100 averaged

Python Code

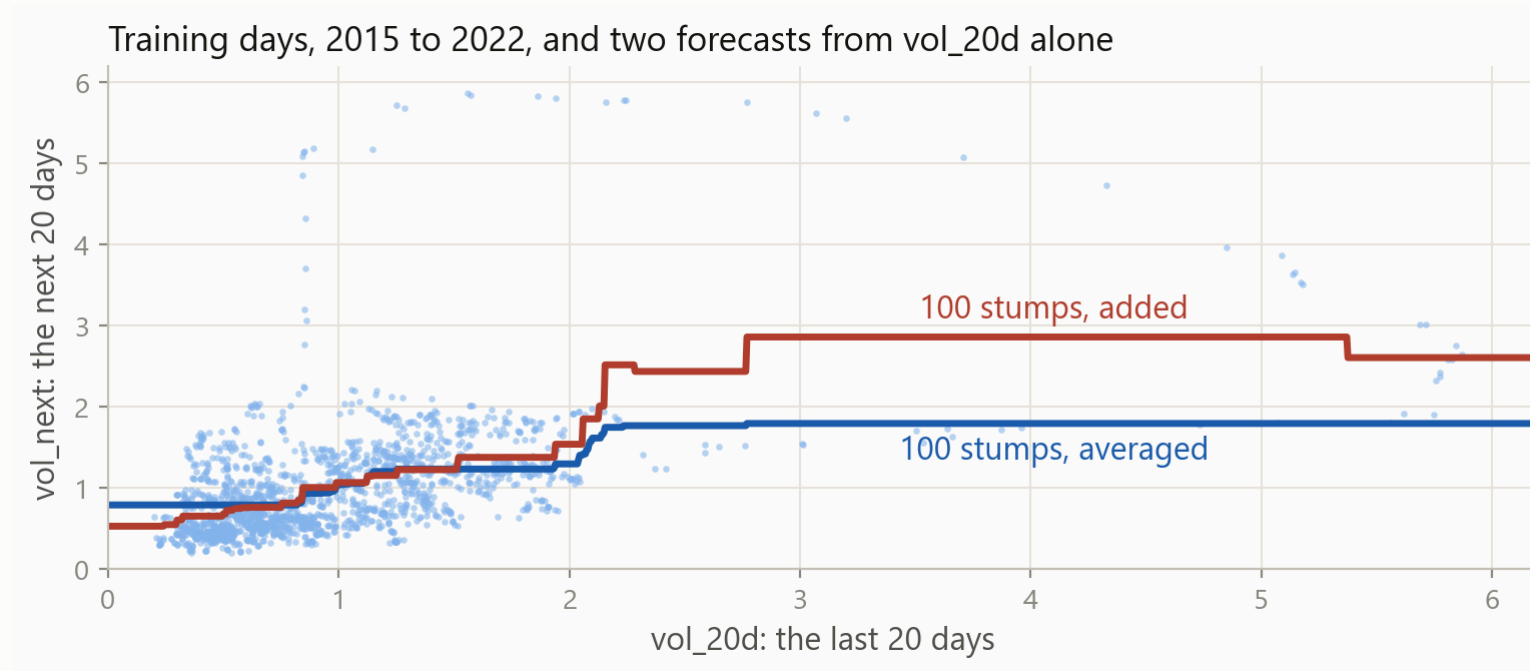
```
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import BaggingRegressor

stump = DecisionTreeRegressor(max_depth=1, random_state=0).fit(train[columns], train["vol_next"])
bagged = BaggingRegressor(DecisionTreeRegressor(max_depth=1), n_estimators=100, random_state=0)
bagged.fit(train[columns], train["vol_next"])
for model in [stump, bagged]:
    print(round(rmse(train["vol_next"], model.predict(train[columns])), 4),
          round(rmse(test["vol_next"], model.predict(test[columns])), 4))
```

```
0.5998 0.2623
0.5771 0.2545
```

One stump errs 0.600 on the training days and 0.262 on the test days. The average of 100 stumps, each grown on a bootstrap sample, barely improves either.

Averaging stumps, and adding them



On `vol_20d` alone, 100 averaged stumps blur one step into a ramp from 0.79 to 1.80. Added one after another, 100 stumps build a staircase from 0.53 to 2.86.

Two ways to combine trees

Averaging: a forest

Deep trees, grown side by side on bootstrap samples, then averaged. Each tree fits its sample closely, so what the average removes is variance.

Adding: boosting

Small trees, grown one after another, each fitted to what the trees before it still get wrong. Each tree is too simple, so what the sum removes is bias.

An average of stumps is about as simple as one stump. A sum of stumps can bend wherever the data does.

2 • Boosting

PART 2

The definition, the procedure, and the procedure run by hand on six days.

Boosting as a formula

$$\hat{f}_M(\mathbf{x}) = \bar{y} + \nu \sum_{m=1}^M h_m(\mathbf{x})$$

Each h_m is a small tree and M is the number of trees; ν , between 0 and 1, is the learning rate. The trees are added, so there is no $1/B$.

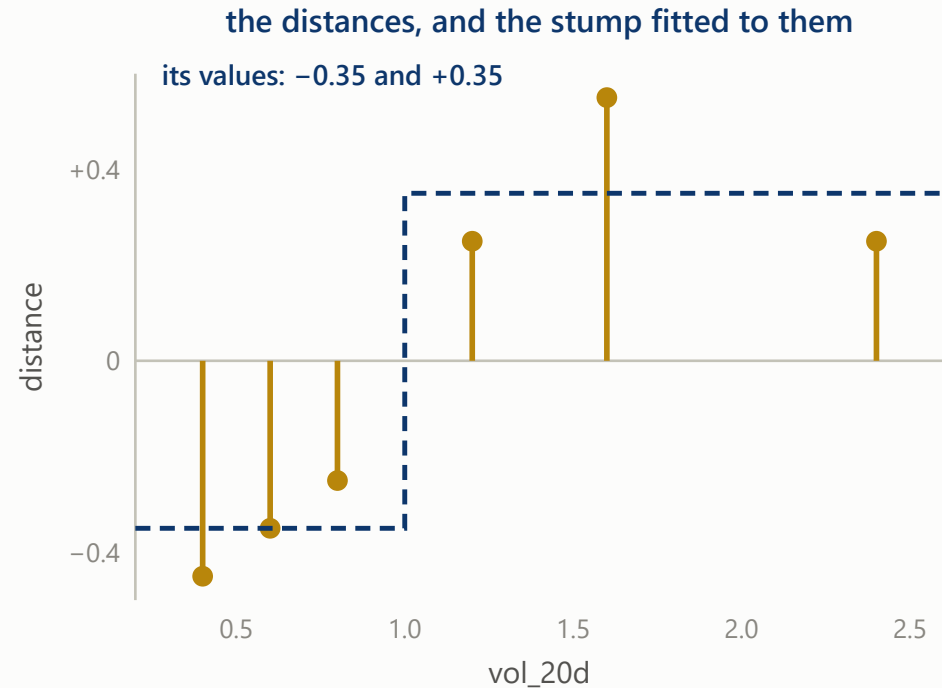
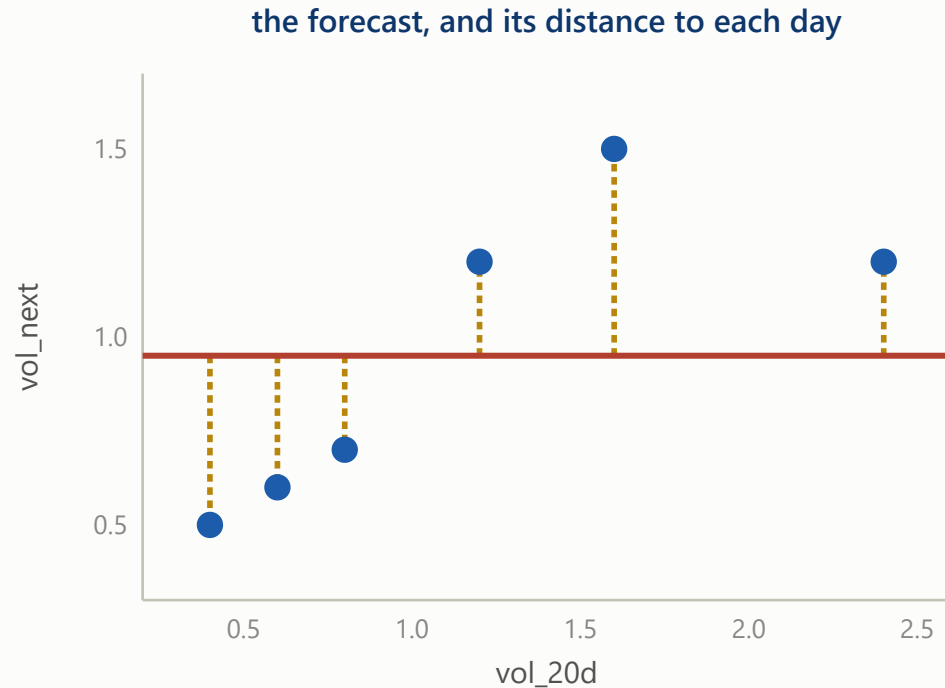
Tree m is fitted to what the first $m - 1$ trees left: the residuals $y_i - \hat{f}_{m-1}(\mathbf{x}_i)$.

The procedure

1. Start every row at the mean: $\hat{f}_0(\mathbf{x}) = \bar{y}$.
2. For each $m = 1, \dots, M$:
 - compute the residuals on the training rows, $r_i = y_i - \hat{f}_{m-1}(\mathbf{x}_i)$;
 - fit a small tree h_m to the pairs $(\mathbf{x}_i, r_i)$;
 - add a step of it: $\hat{f}_m(\mathbf{x}) = \hat{f}_{m-1}(\mathbf{x}) + \nu h_m(\mathbf{x})$.
3. Forecast new rows with $\hat{f}_M$.

Three settings shape the result: the number of trees M , the learning rate ν , and the size of each tree.

Boosting the six days, step by step

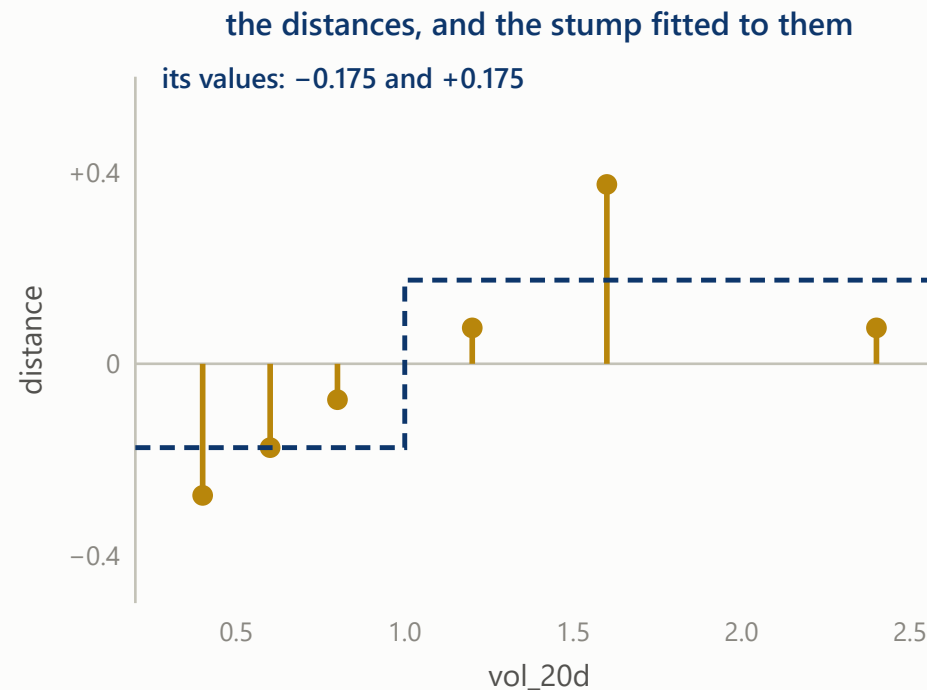
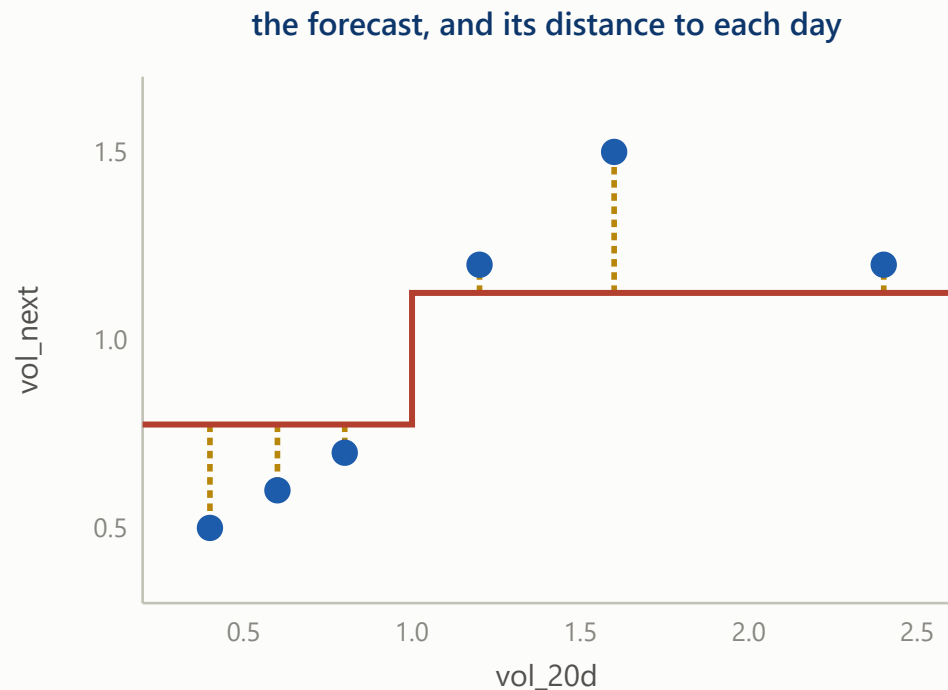


Start: every day gets the mean, 0.95. A stump fitted to the distances cuts after 0.8.

RSS 0.815

The six days from last time, with $\nu = 0.5$: each step fits a stump to the dashed distances and adds half of it. Six stumps take the RSS from 0.815 to 0.038.

Boosting the six days, step by step

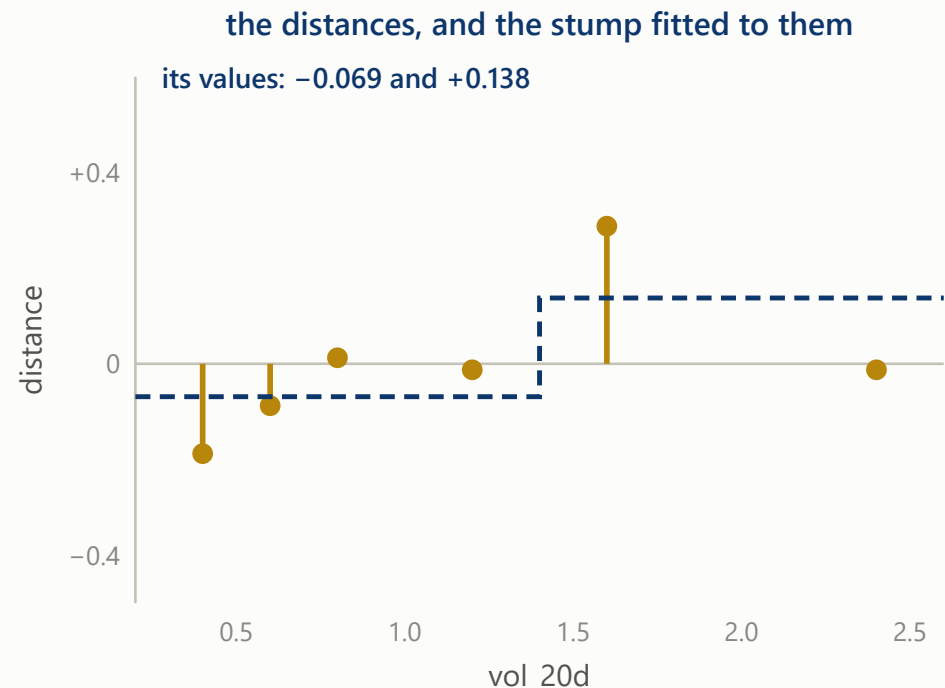
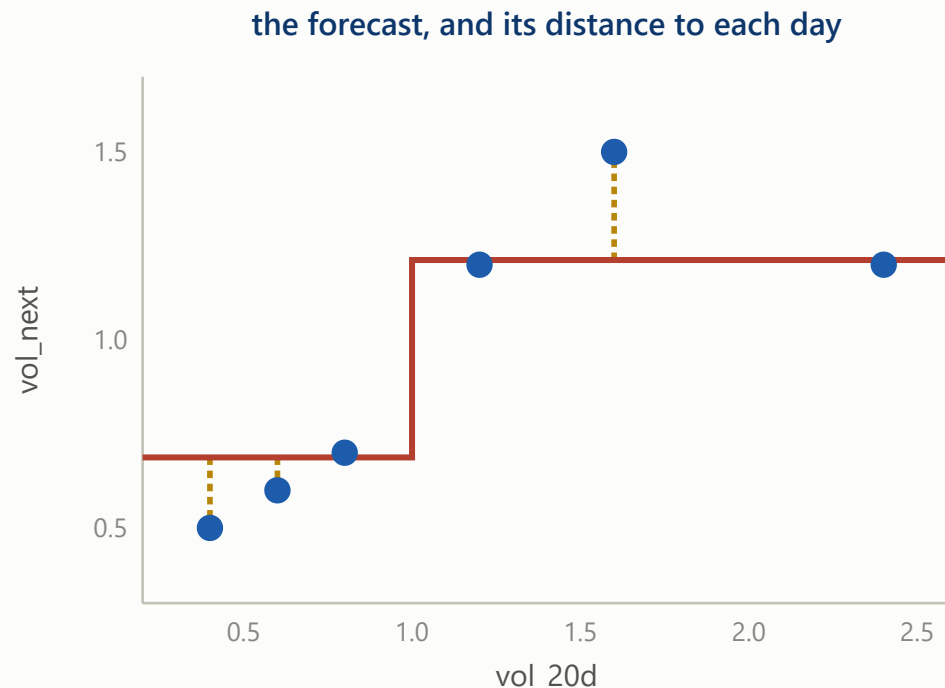


Half of that stump is added. Half of each gap is left, so the next stump cuts after 0.8 again.

1 stump · RSS 0.264

The six days from last time, with $\nu = 0.5$: each step fits a stump to the dashed distances and adds half of it. Six stumps take the RSS from 0.815 to 0.038.

Boosting the six days, step by step

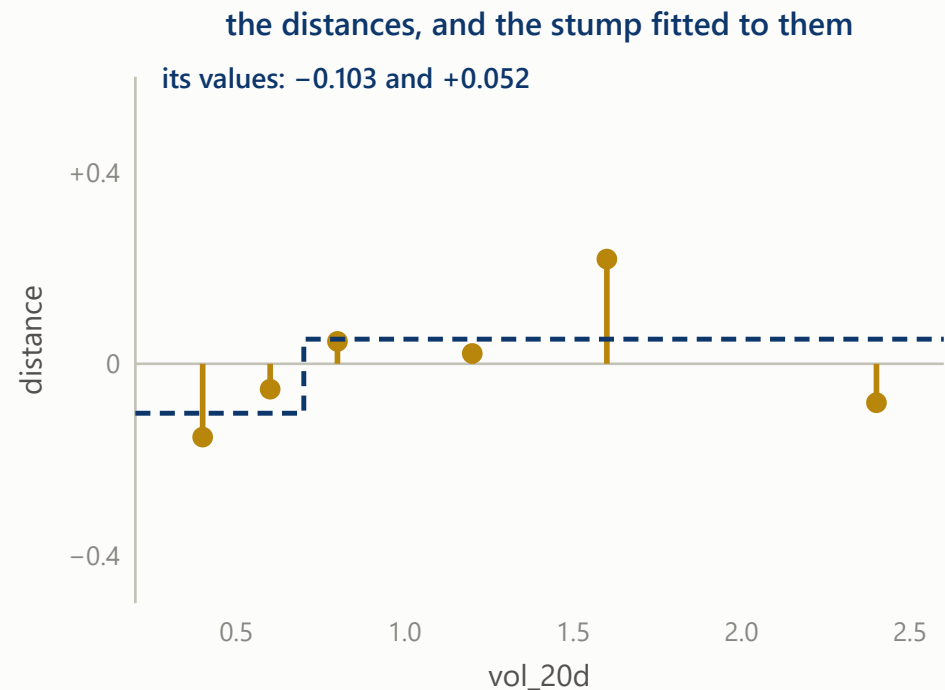
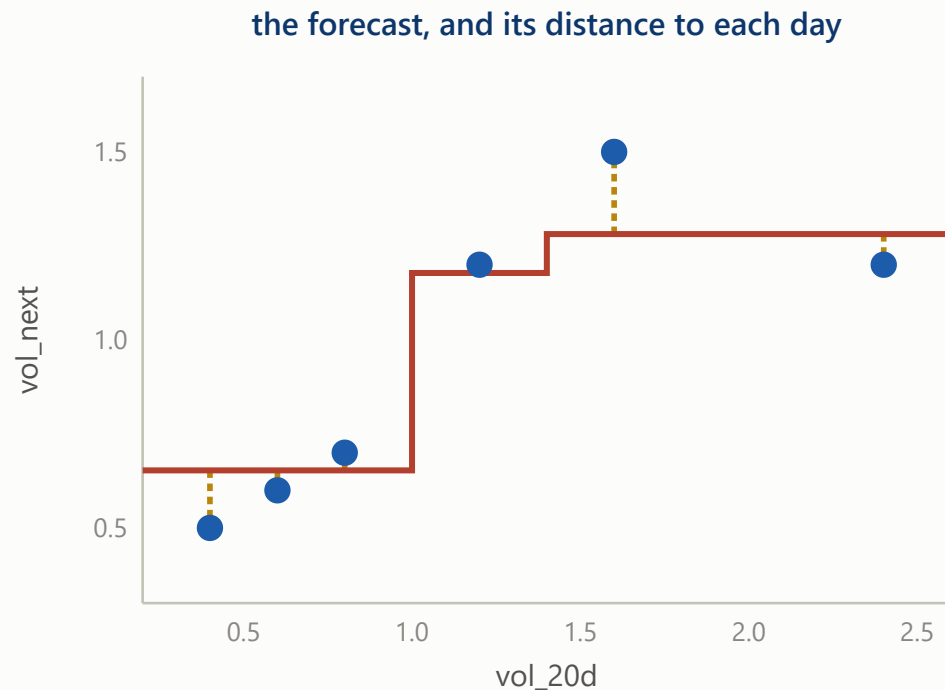


The day at 1.6 is now furthest off, and the next stump cuts after 1.2.

2 stumps · RSS 0.126

The six days from last time, with $\nu = 0.5$: each step fits a stump to the dashed distances and adds half of it. Six stumps take the RSS from 0.815 to 0.038.

Boosting the six days, step by step

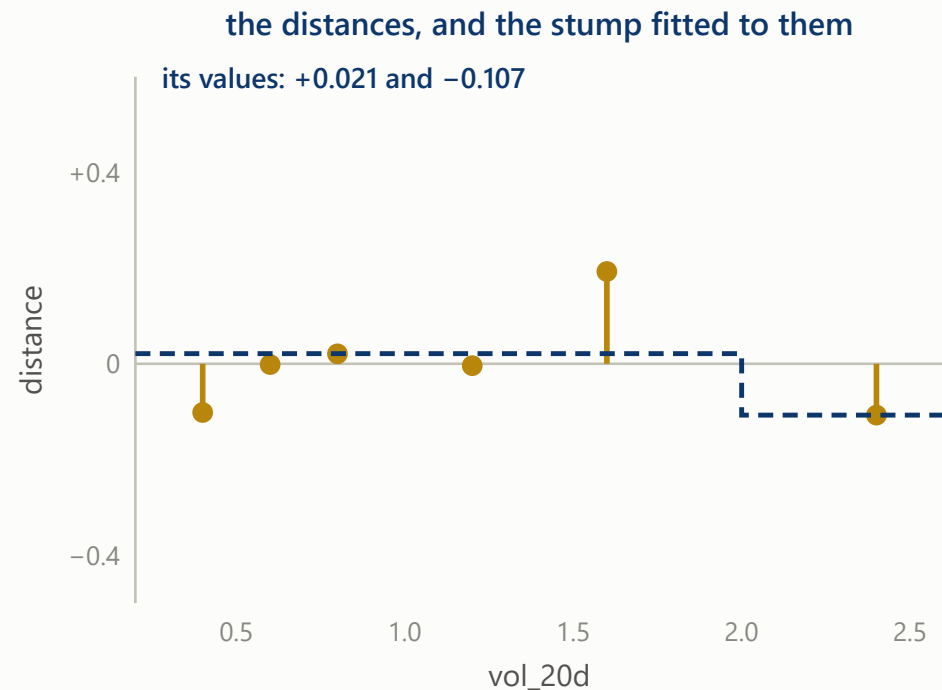
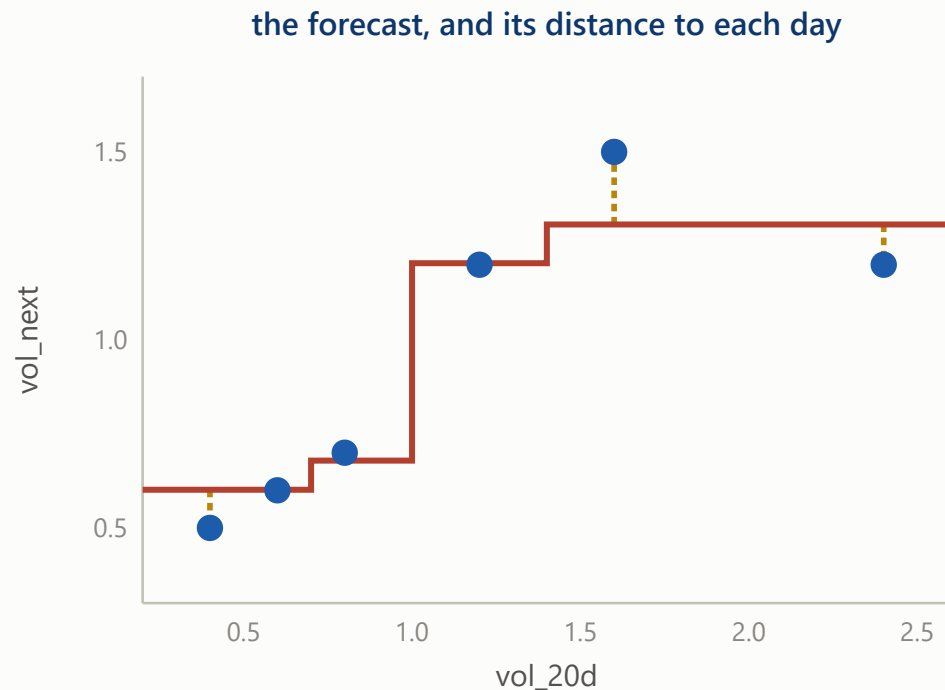


The two calmest days are too high, and the next stump cuts after 0.6.

3 stumps · RSS 0.083

The six days from last time, with $\nu = 0.5$: each step fits a stump to the dashed distances and adds half of it. Six stumps take the RSS from 0.815 to 0.038.

Boosting the six days, step by step

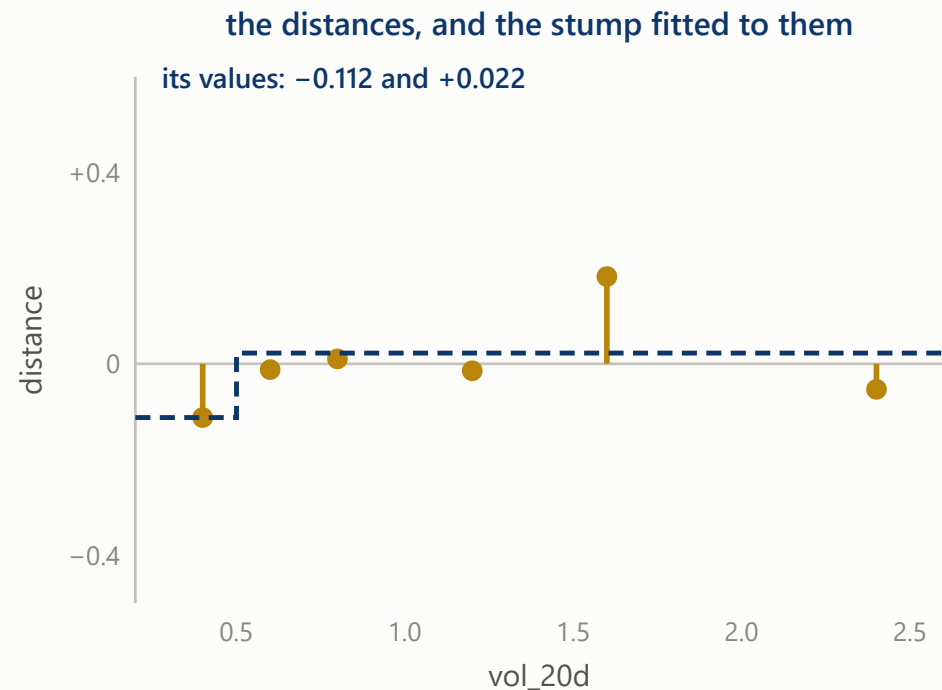
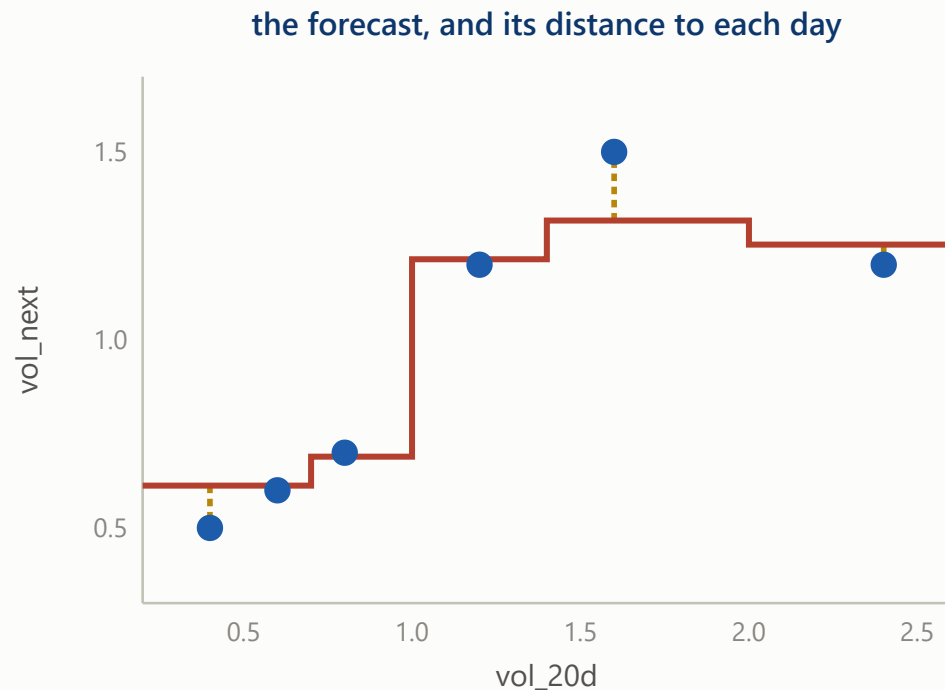


The day at 2.4 is too high, and the next stump cuts after 1.6.

4 stumps · RSS 0.059

The six days from last time, with $\nu = 0.5$: each step fits a stump to the dashed distances and adds half of it. Six stumps take the RSS from 0.815 to 0.038.

Boosting the six days, step by step



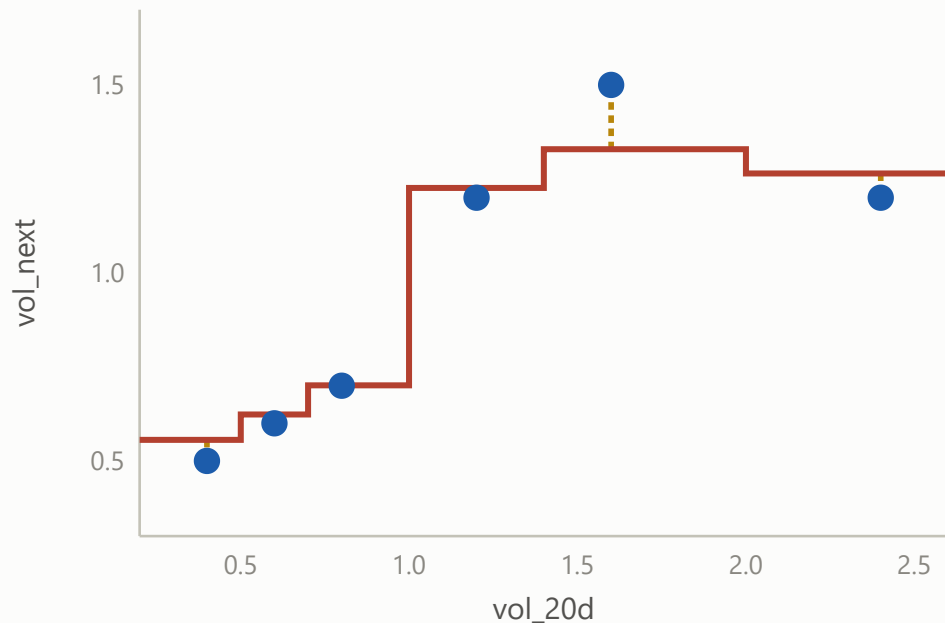
The calmest day is too high, and the next stump cuts after 0.4.

5 stumps · RSS 0.049

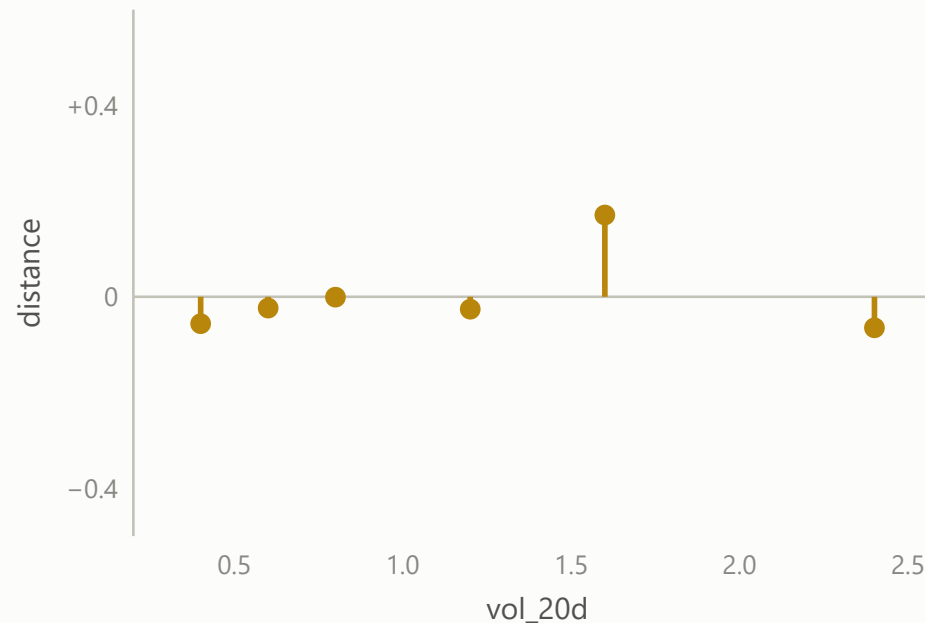
The six days from last time, with $\nu = 0.5$: each step fits a stump to the dashed distances and adds half of it. Six stumps take the RSS from 0.815 to 0.038.

Boosting the six days, step by step

the forecast, and its distance to each day



the distances, and the stump fitted to them



After six stumps every forecast is within 0.17 of its day.

6 stumps • RSS 0.038

The six days from last time, with $\nu = 0.5$: each step fits a stump to the dashed distances and adds half of it. Six stumps take the RSS from 0.815 to 0.038.

The six, in code

Python Code

```
six = pd.DataFrame({"vol_20d": [0.4, 0.6, 0.8, 1.2, 1.6, 2.4],  
                    "vol_next": [0.5, 0.6, 0.7, 1.2, 1.5, 1.2]})  
forecast = np.full(6, six["vol_next"].mean())  
for step in range(6):  
    h = DecisionTreeRegressor(max_depth=1).fit(six[["vol_20d"]], six["vol_next"] - forecast)  
    forecast = forecast + 0.5 * h.predict(six[["vol_20d"]])  
    print(round(((six["vol_next"] - forecast) ** 2).sum(), 4))
```

```
0.2638  
0.1259  
0.0834  
0.0595  
0.0492  
0.0378
```

Your turn 1

Run the loop for 50 steps instead of 6. What happens to the RSS, and to the forecast?

Python Code

```
forecast = np.full(6, six["vol_next"].mean())
for step in range(...):
    h = DecisionTreeRegressor(max_depth=1).fit(six[["vol_20d"]], six["vol_next"] - forecast)
    forecast = forecast + 0.5 * h.predict(six[["vol_20d"]])
print(round((((six["vol_next"] - forecast) ** 2).sum(), 6))
print(forecast.round(3))
```

The number of steps goes inside `range`. Compare the forecast with `six["vol_next"]`.

Your turn 1: answer

```
for step in range(50):
```

The RSS falls to 0.0001, and every forecast is within 0.006 of its day: 50 stumps have learned the six days. On new days, that is overfitting.

Adding trees can always lower the training error, so the number of trees has to be chosen on other rows.

Residuals as a gradient

$$L_i = \frac{1}{2} (y_i - \hat{f}(\mathbf{x}_i))^2 \implies -\frac{\partial L_i}{\partial \hat{f}(\mathbf{x}_i)} = y_i - \hat{f}(\mathbf{x}_i)$$

The residual is the negative gradient of the squared error with respect to the forecast. Each tree is one step of gradient descent on the forecasts, hence the name gradient boosting.

With another loss, each tree is fitted to that loss's negative gradient instead: for log-loss, $y_i - \hat{p}_i$.

Boosting by hand, on the table

Python Code

```
forecast = np.full(len(train), train["vol_next"].mean())
test_forecast = np.full(len(test), train["vol_next"].mean())
for m in range(100):
    h = DecisionTreeRegressor(max_depth=1, random_state=0)
    h.fit(train[columns], train["vol_next"] - forecast)
    forecast = forecast + 0.1 * h.predict(train[columns])
    test_forecast = test_forecast + 0.1 * h.predict(test[columns])
print(round(rmse(test["vol_next"], test_forecast), 4))
```

0.2221

100 stumps on the 19 columns, each fitted to what is left and added at $\nu = 0.1$. The test error is 0.222, where 100 averaged stumps scored 0.255.

Gradient boosting in scikit-learn

Python Code

```
from sklearn.ensemble import GradientBoostingRegressor

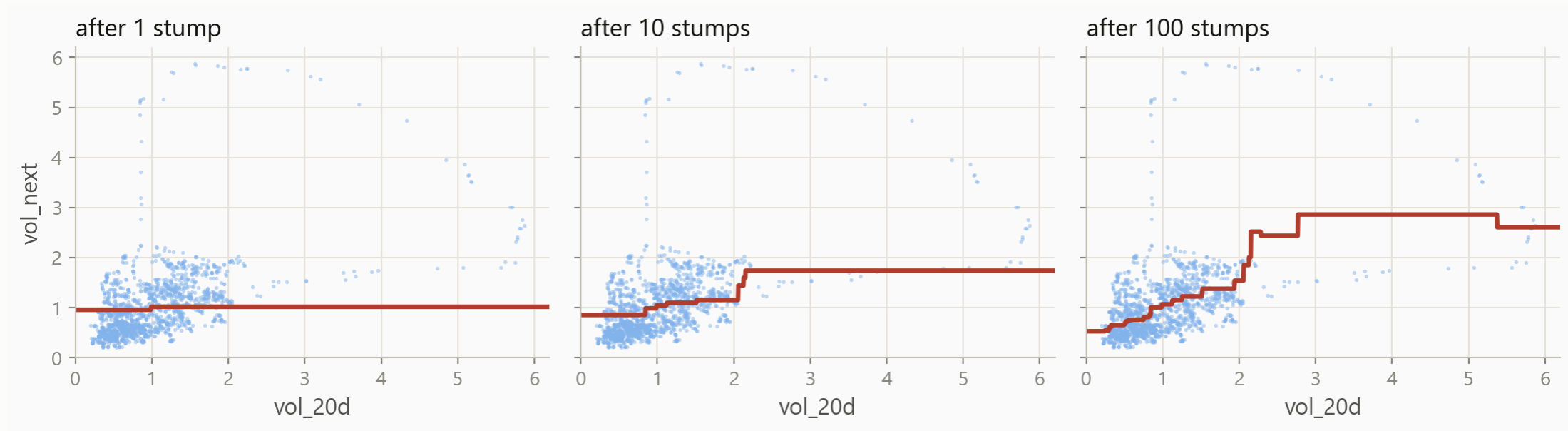
boost = GradientBoostingRegressor(n_estimators=100, learning_rate=0.1, max_depth=1, random_state=0)
boost.fit(train[columns], train["vol_next"])
print(round(rmse(train["vol_next"], boost.predict(train[columns])), 4),
      round(rmse(test["vol_next"], boost.predict(test[columns])), 4))
print(np.allclose(boost.predict(test[columns]), test_forecast))
```

0.4569 0.2221

True

`n_estimators` is M , `learning_rate` is ν , and `max_depth=1` makes each tree a stump. The forecasts are the loop's: 0.457 on the training days, 0.222 on the test days.

The forecast after 1, 10 and 100 stumps



The first stump moves the forecast a tenth of the way: 0.96 to 1.02. After 10 stumps it spans 0.86 to 1.74, and after 100, 0.53 to 2.86.

3 • How many trees

PART 3

Every tree lowers the training error. On new days, too many trees do harm.

1,000 stumps **PREDICT**

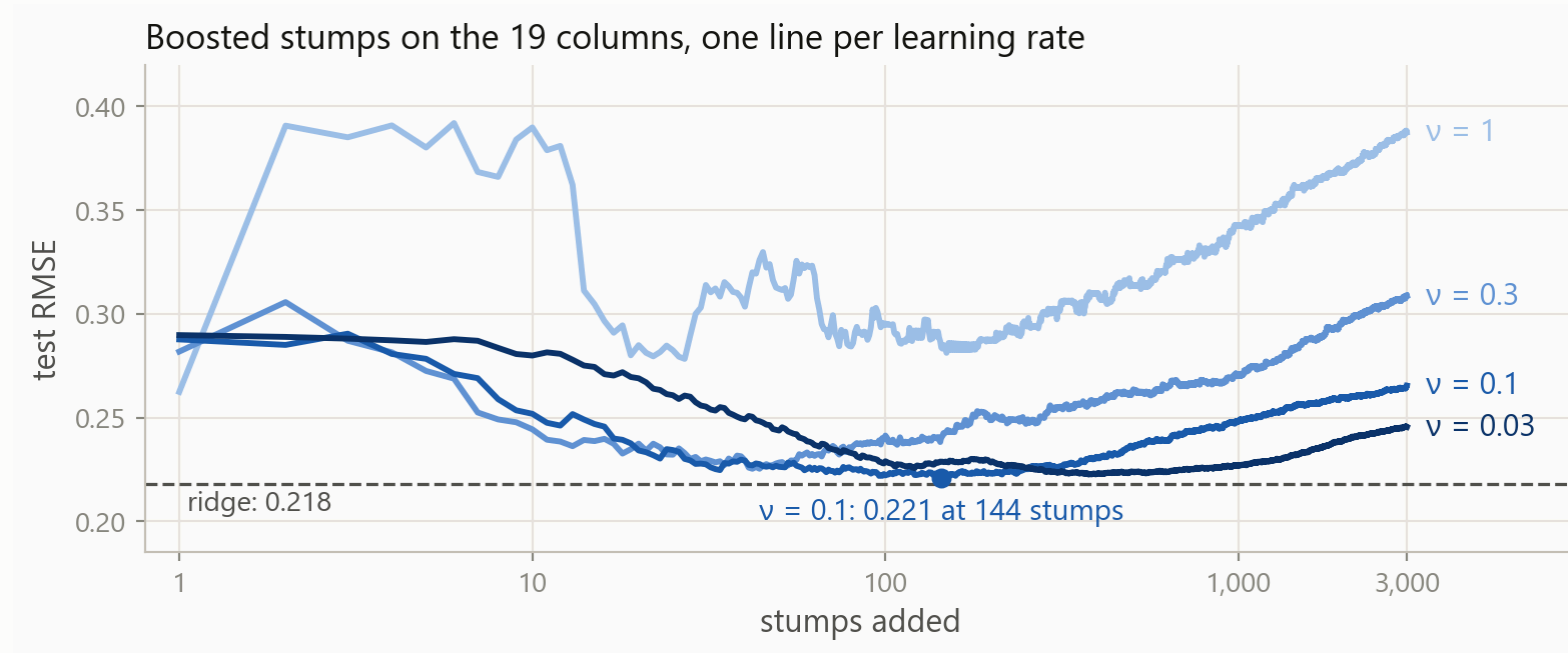
A forest's test error only flattened as trees were added. Fit ten times as many stumps as **boost**: what will the test error do?

Python Code

```
many = GradientBoostingRegressor(n_estimators=1000, learning_rate=0.1, max_depth=1, random_state=0)
many.fit(train[columns], train["vol_next"])

print(round(rmse(train["vol_next"], many.predict(train[columns])), 4))
print(round(rmse(test["vol_next"], many.predict(test[columns])), 4))
```

Test error against the number of trees



1,000 stumps err 0.350 on the training days and 0.248 on the test days. At every learning rate the test error falls, then rises; at 0.1 it is lowest near 144.

Your turn 2

Grow trees of depth 2 instead of stumps, with everything else as in **boost**. Better or worse on the test days?

Python Code

```
deeper = GradientBoostingRegressor(n_estimators=100, learning_rate=0.1, max_depth=..., random_state=0)
deeper.fit(train[columns], train["vol_next"])
print(round(rmse(train["vol_next"], deeper.predict(train[columns])), 4))
print(round(rmse(test["vol_next"], deeper.predict(test[columns])), 4))
```

max_depth sets the questions on each path, as for one tree. Stumps scored 0.457 and 0.222.

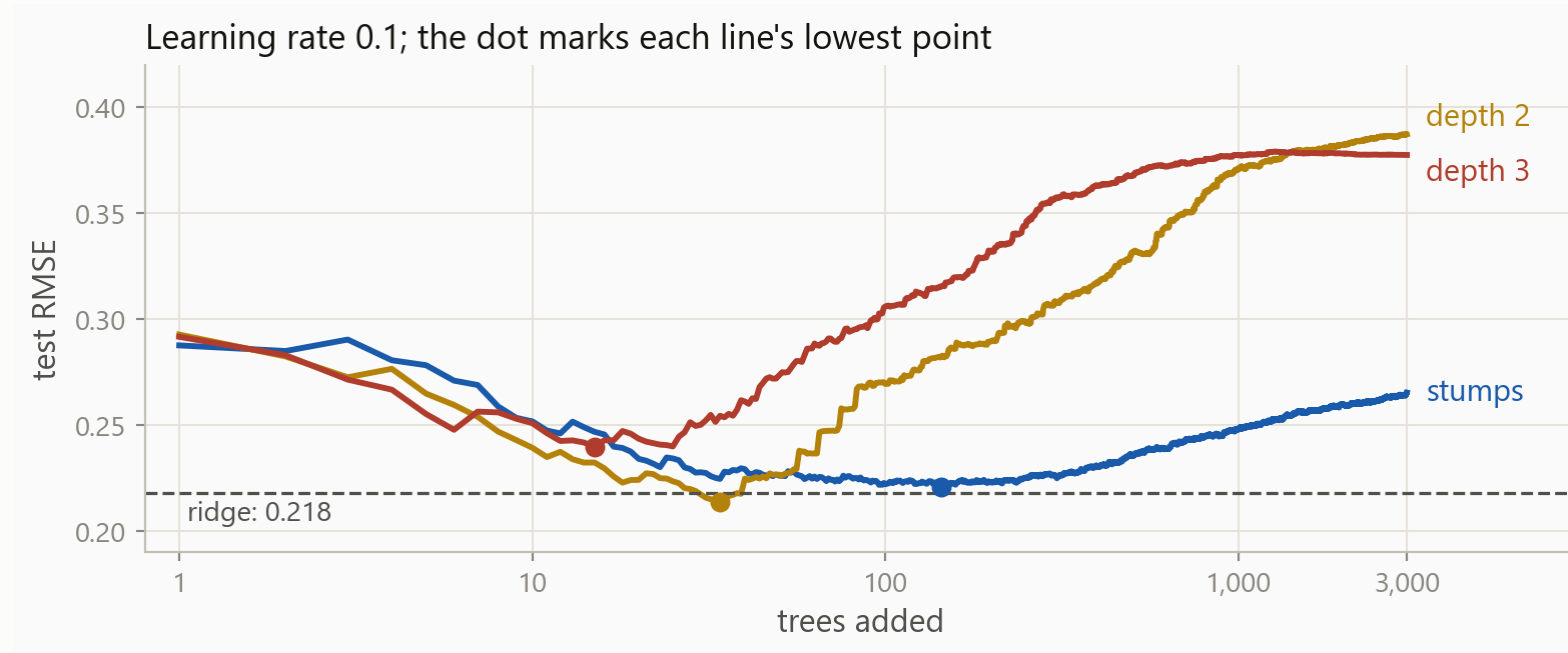
Your turn 2: answer

```
deeper = GradientBoostingRegressor(n_estimators=100, learning_rate=0.1, max_depth=2, random_state=0)
```

Trees of depth 2 fit the training days more closely, 0.299 against 0.457, and the test days worse, 0.269 against 0.222.

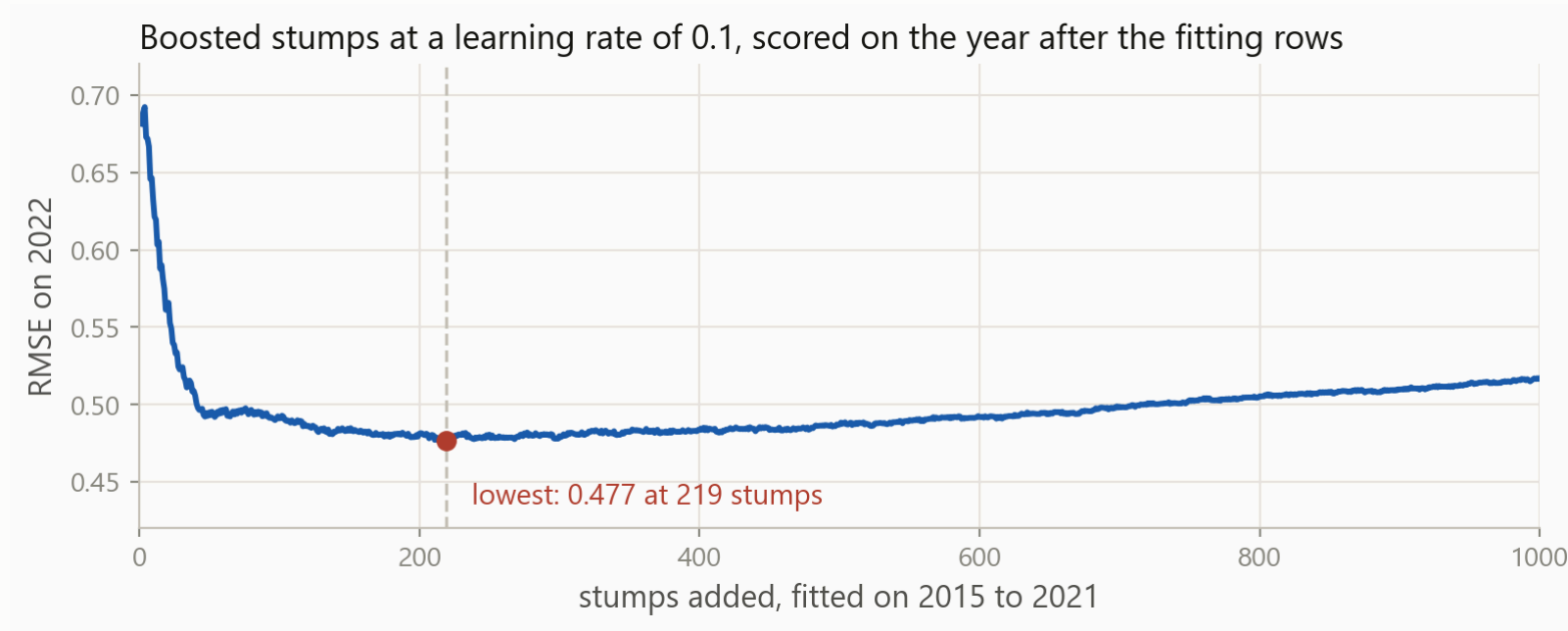
A tree of depth 2 can fit how two columns interact, and 100 of them also fit the noise in the training days.

The depth of each tree



Deeper trees reach their lowest test error sooner and then climb faster: after 15 trees at depth 3, 34 at depth 2 and 144 with stumps.

Stopping early



Fitted on 2015 to 2021 and scored on 2022 after every stump, the error is lowest at 219. That model scores 0.217 on the test days; ridge scored 0.218.

Early stopping on random days

Python Code

```
early = GradientBoostingRegressor(n_estimators=5000, learning_rate=0.1, max_depth=1,  
                                  validation_fraction=0.1, n_iter_no_change=10, random_state=0)  
early.fit(train[columns], train["vol_next"])  
print(early.n_estimators_, round(rmse(test["vol_next"], early.predict(test[columns])), 4))
```

342 0.2263

`n_iter_no_change=10` stops once 10 more trees have not helped on a random tenth of the training days: at 342 trees, with 0.226 on the test days.

A random day has its neighbours among the fitting rows, as out-of-bag days did, so the search stops late. On days, stop on a later block.

4 • XGBoost and LightGBM

PART 4

The two libraries most used for boosting in practice.

Two libraries built for boosting

XGBoost

Chen and Guestrin (2016). Bins each column, puts a penalty on every tree, handles missing values and uses every core. In 2015, 17 of the 29 winning solutions on Kaggle's blog used it.

LightGBM

Ke and others, at Microsoft (2017). Bins each column and grows each tree leaf by leaf, to be fast on large tables. Both libraries follow scikit-learn's pattern: create, fit, predict.

On 45 tables, tree-based models stayed ahead of deep learning on medium-sized data, about 10,000 rows (Grinsztajn, Oyallon and Varoquaux, 2022).

XGBoost in code

Python Code

```
from xgboost import XGBRegressor

xgb = XGBRegressor(n_estimators=100, learning_rate=0.1, max_depth=1, random_state=0)
xgb.fit(train[columns], train["vol_next"])
print(round(rmse(test["vol_next"], xgb.predict(test[columns])), 4))
```

0.2194

The same 100 stumps score 0.219, since XGBoost searches for cuts among 256 bins per column. Its defaults, 100 trees of depth 6 at a rate of 0.3, score 0.349.

A penalty on each tree

$$\sum_{i=1}^n L(y_i, \hat{f}(\mathbf{x}_i)) + \sum_{m=1}^M \left(\gamma T_m + \frac{1}{2} \lambda \sum_{j=1}^{T_m} w_{mj}^2 \right)$$

T_m counts the leaves of tree m and w_{mj} are their values. `reg_lambda` is λ , a ridge penalty on the values; `gamma` is γ , a price per leaf, as in pruning.

The defaults are `reg_lambda=1` and `gamma=0`: a light ridge penalty on the leaves, and no price for adding one.

LightGBM in code

Python Code

```
from lightgbm import LGBMRegressor

lgbm = LGBMRegressor(n_estimators=100, learning_rate=0.1, num_leaves=2, random_state=0, verbose=-1)
lgbm.fit(train[columns], train["vol_next"])
print(round(rmse(test["vol_next"], lgbm.predict(test[columns])), 4))
```

0.2255

LightGBM sizes a tree by its leaves, so **num_leaves=2** makes a stump: 0.226. Its default of 31 leaves scores 0.346, and **verbose=-1** silences its messages.

Depth by depth, or leaf by leaf

depth by depth

XGBoost, scikit-learn's GradientBoosting

leaf

leaf by leaf

LightGBM, scikit-learn's HistGradientBoosting

leaf

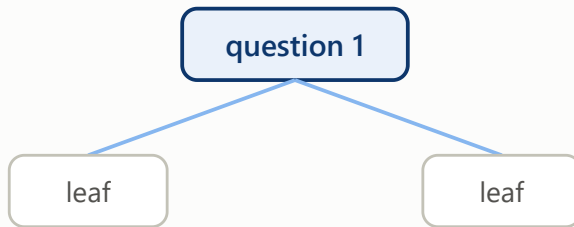
Both start with every row in one box.

max_depth limits a tree grown depth by depth, and **num_leaves** one grown leaf by leaf. With the same leaves, a leaf-by-leaf tree goes deeper where the error is.

Depth by depth, or leaf by leaf

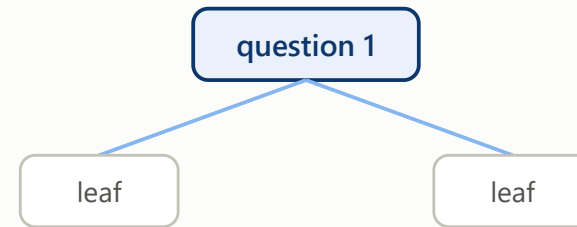
depth by depth

XGBoost, scikit-learn's GradientBoosting



leaf by leaf

LightGBM, scikit-learn's HistGradientBoosting



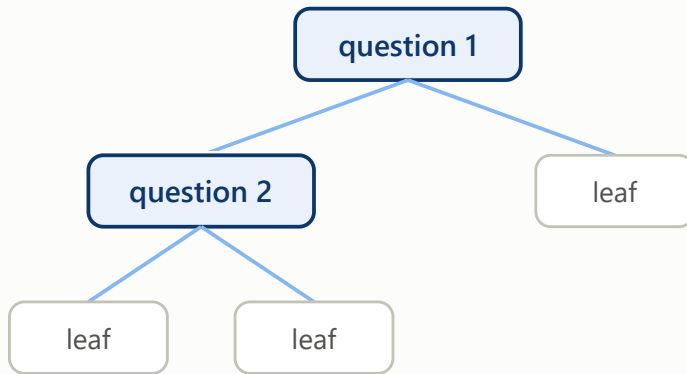
Step 1: both ask a first question.

max_depth limits a tree grown depth by depth, and **num_leaves** one grown leaf by leaf. With the same leaves, a leaf-by-leaf tree goes deeper where the error is.

Depth by depth, or leaf by leaf

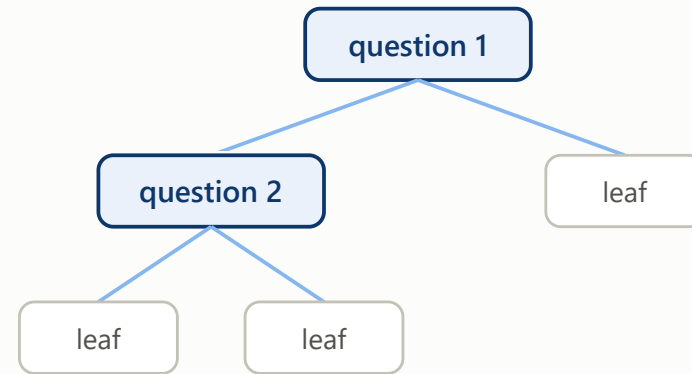
depth by depth

XGBoost, scikit-learn's GradientBoosting



leaf by leaf

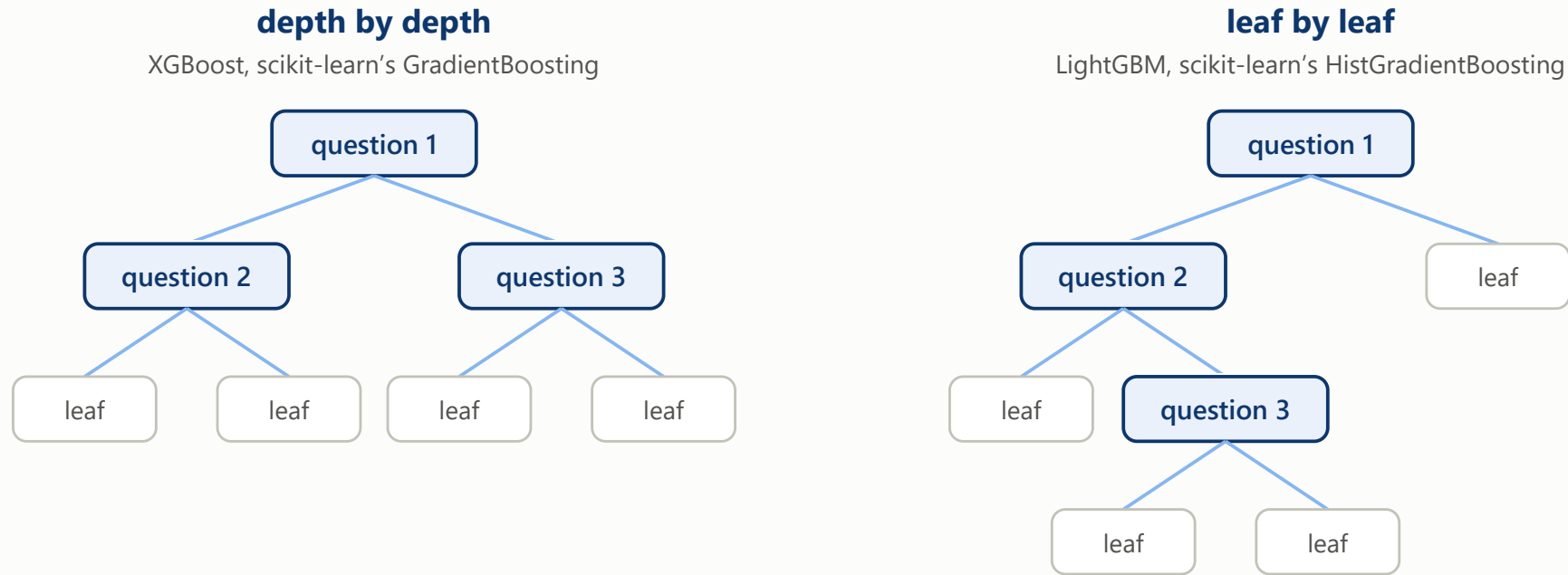
LightGBM, scikit-learn's HistGradientBoosting



Step 2: both split the left box.

max_depth limits a tree grown depth by depth, and **num_leaves** one grown leaf by leaf. With the same leaves, a leaf-by-leaf tree goes deeper where the error is.

Depth by depth, or leaf by leaf



Step 3: depth by depth splits the right box to finish depth 2.
Leaf by leaf splits whichever leaf lowers the error most, here one level deeper.

max_depth limits a tree grown depth by depth, and **num_leaves** one grown leaf by leaf. With the same leaves, a leaf-by-leaf tree goes deeper where the error is.

Early stopping in XGBoost

Python Code

```
fit_rows, stop_rows = train.loc[:"2021-12-31"], train.loc["2022-01-01":]  
xgb_stop = XGBRegressor(n_estimators=3000, learning_rate=0.1, max_depth=1,  
                        early_stopping_rounds=50, random_state=0)  
xgb_stop.fit(fit_rows[columns], fit_rows["vol_next"],  
            eval_set=[(stop_rows[columns], stop_rows["vol_next"])], verbose=False)  
print(xgb_stop.best_iteration, round(rmse(test["vol_next"], xgb_stop.predict(test[columns])), 4))
```

243 0.2246

XGBoost scores every new tree on 2022 and stops 50 trees after the best.

best_iteration counts from 0, so **predict** uses the first 244 stumps: 0.225.

Your turn 3

Suppose early stopping watched the test days instead of 2022. Pass them as **eval_set**: which score does it report, and is that a test score?

Python Code

```
xgb_peek = XGBRegressor(n_estimators=3000, learning_rate=0.1, max_depth=1,
                        early_stopping_rounds=50, random_state=0)
xgb_peek.fit(fit_rows[columns], fit_rows["vol_next"],
            eval_set=[(..., ...)], verbose=False)
print(xgb_peek.best_iteration, round(xgb_peek.best_score, 4))
```

eval_set takes a list of (columns, target) pairs, as in **xgb_stop**. **best_score** is the lowest RMSE early stopping saw.

Your turn 3: answer

```
xgb_peek.fit(fit_rows[columns], fit_rows["vol_next"],  
             eval_set=[(test[columns], test["vol_next"])], verbose=False)
```

It keeps 30 stumps and reports 0.204, which would beat ridge's 0.218. It is not a test score: the test days chose when to stop, out of 80 scores computed on them.

Rows that help choose a setting become validation rows. The test days report once, for a model chosen without them: here 0.225.

5 • Boosting a label

PART 5

The credit table, where trees beat logistic regression.

The credit table

Python Code

```
from sklearn.model_selection import train_test_split

credit = pd.read_csv("data/credit.csv")
c_train, c_test = train_test_split(credit, test_size=0.3, random_state=0,
                                   stratify=credit["default"])
c_columns = ["limit", "age", "late_now", "months_late", "bill", "paid", "utilisation"]
print(len(c_train), len(c_test), round(c_train["default"].mean(), 3))
```

```
21000 9000 0.221
```

One row per card holder; **default** is 1 for the 22 percent who defaulted the next month. On this split logistic regression scored 0.748 and the forest 0.777.

Boosting a label

$$\hat{p}_i = \sigma(\hat{f}(\mathbf{x}_i)), \quad -\frac{\partial \log\text{-loss}_i}{\partial \hat{f}(\mathbf{x}_i)} = y_i - \hat{p}_i$$

The trees are added on the log-odds scale, and each is fitted to $y_i - \hat{p}_i$, the gap between the label and the probability. The sigmoid turns the sum into a probability.

XGBoost on the credit table

Python Code

```
from xgboost import XGBClassifier
from sklearn.metrics import roc_auc_score

xgb_c = XGBClassifier(n_estimators=100, learning_rate=0.1, max_depth=3, random_state=0)
xgb_c.fit(c_train[c_columns], c_train["default"])
p_xgb = xgb_c.predict_proba(c_test[c_columns])[:, 1]
print(round(roc_auc_score(c_test["default"], p_xgb), 4))
```

0.777

100 trees of depth 3 at a learning rate of 0.1: an AUC of 0.777, the forest's score, against 0.748 for logistic regression.

Four implementations of boosting

model	how a cut is searched	fit, on one core	test AUC
GradientBoostingClassifier	every value of every column	1.91 s	0.776
HistGradientBoostingClassifier	255 bins per column	0.14 s	0.776
XGBClassifier	256 bins per column	0.08 s	0.777
LGBMClassifier	255 bins, leaf by leaf	0.08 s	0.777

100 trees of depth 3, or of 8 leaves, at a learning rate of 0.1, on the 21,000 training rows. With bins, the same trees fit about 25 times faster.

Early stopping on separate borrowers

Python Code

```
from lightgbm import LGBMClassifier, early_stopping

fit_c, stop_c = train_test_split(c_train, test_size=0.2, random_state=0, stratify=c_train["default"])
lgbm_c = LGBMClassifier(n_estimators=3000, learning_rate=0.05, num_leaves=15, random_state=0, verbose=-1)
lgbm_c.fit(fit_c[c_columns], fit_c["default"], eval_set=[(stop_c[c_columns], stop_c["default"])],
          eval_metric="auc", callbacks=[early_stopping(50, verbose=False)])
p_lgbm = lgbm_c.predict_proba(c_test[c_columns])[:, 1]
print(lgbm_c.best_iteration_, round(roc_auc_score(c_test["default"], p_lgbm), 4))
```

91 0.7757

Borrowers are separate cases, so a random fifth of the training rows can decide when to stop: 91 trees of 15 leaves, and an AUC of 0.776.

When to change what

Always

`n_estimators`, found by early stopping, with
`learning_rate`: a smaller rate needs more trees. The size of each tree: `max_depth`, or `num_leaves` in LightGBM.

Rarely

`gamma`, a price per leaf. `max_bin`, the number of bins per column. `objective`, the loss that is boosted.

Sometimes

`subsample` and `colsample_bytree`, a random share of the rows and columns for each tree, as in a forest; LightGBM also needs `subsample_freq=1`. The smallest leaf, `min_child_weight` or `min_child_samples`.
`reg_lambda`.

Never needed

A scaler. A cut compares one column with one number, so scaling changes no forecast.

6 • Reading a model

PART 6

Which columns a model of many trees uses, measured two ways.

Importance from the cuts

Python Code

```
importance = pd.Series(xgb_c.feature_importances_, index=c_columns)
print(importance.sort_values(ascending=False).round(3))
```

```
late_now      0.564
months_late   0.307
paid          0.034
bill          0.029
limit         0.027
utilisation    0.026
age           0.011
dtype: float32
```

A column of random numbers

Python Code

```
noisy_train = c_train.copy()
noisy_train["noise"] = np.random.default_rng(0).normal(size=len(noisy_train))
n_columns = c_columns + ["noise"]
noisy_model = LGBMClassifier(random_state=0, verbose=-1)
noisy_model.fit(noisy_train[n_columns], noisy_train["default"])
cuts = pd.Series(noisy_model.feature_importances_, index=n_columns)
print(cuts.sort_values(ascending=False).to_dict())
```

```
{'noise': 582, 'utilisation': 512, 'bill': 493, 'paid': 462, 'age': 415, 'limit': 305,
 'months_late': 148, 'late_now': 83}
```

LightGBM counts the cuts on each column. The noise gets the most, 582: each of its 21,000 values is a possible cut, where **late_now** has 9 values and gets 83.

What each library counts

model	what <code>feature_importances_</code> measures
scikit-learn: trees, forests, <code>GradientBoosting</code>	the fall in squared error or Gini from each column's cuts, summed and scaled to one
XGBoost	the average fall in the loss per cut on the column, scaled to one
LightGBM	the number of cuts on the column; <code>importance_type="gain"</code> sums the fall instead
scikit-learn: <code>HistGradientBoosting</code>	it has no <code>feature_importances_</code>

All four measure on the training rows, where a flexible model has fitted noise too. LightGBM's gain still gives the random column 8 percent, more than `age`.

Permutation importance

$$I_j = s - \frac{1}{K} \sum_{k=1}^K s_{k,j}$$

s is the model's score on the test rows and $s_{k,j}$ its score after the k -th shuffle of column j . Shuffling breaks the column's link to the label and keeps its values.

It asks what the model loses without the column, on rows it did not fit.

Permutation importance in code

Python Code

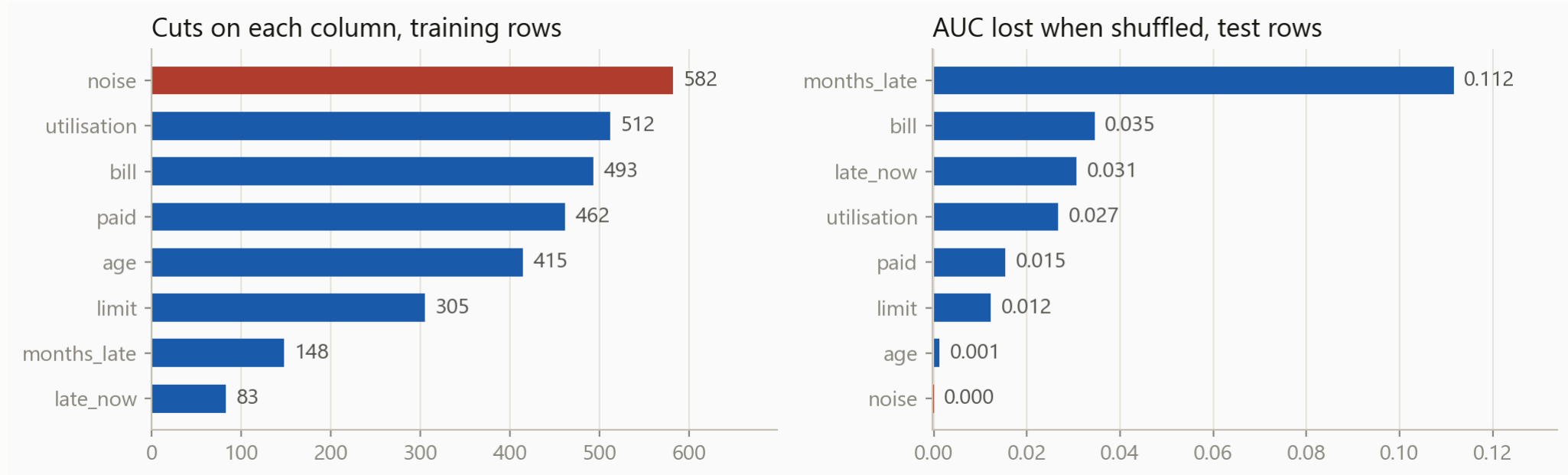
```
from sklearn.inspection import permutation_importance

noisy_test = c_test.copy()
noisy_test["noise"] = np.random.default_rng(1).normal(size=len(noisy_test))
result = permutation_importance(noisy_model, noisy_test[n_columns], noisy_test["default"],
                               scoring="roc_auc", n_repeats=10, random_state=0)
shuffled = pd.Series(result.importances_mean, index=n_columns)
print(shuffled.sort_values(ascending=False).round(4).to_dict())
```

```
{'months_late': 0.1116, 'bill': 0.0345, 'late_now': 0.0306, 'utilisation': 0.0267, 'paid':
0.0153, 'limit': 0.0122, 'age': 0.0013, 'noise': -0.0003}
```

Shuffling **months_late** costs 0.112 of AUC, and shuffling the noise costs nothing. The column the cut counts ranked first comes last.

Two rankings of one model



The noise is shown in brick. Counts of cuts reward columns with many possible cuts on the rows the model fitted; the shuffles are scored on the test rows.

Two correlated columns

column	AUC lost when shuffled	AUC lost by a refit without it
late_now	0.031	0.012
months_late	0.112	0.018
both, together	0.206	0.109

The two columns have a correlation of 0.72. A refit without either one loses little, because the other carries much of the same information.

Importance is what this model loses without a column, given the others. It is not the column's effect on default.

7 • Choosing a model

PART 7

Every model of the course on both tables, what each is for, and the routine they share.

Both tables, every model

model	volatility: test RMSE	credit: test AUC
linear regression on vol_20d; logistic regression	0.237	0.748
ridge	0.218	
lasso	0.217	
k-nearest neighbours		0.749
a tree	0.241	0.722
a random forest	0.226	0.777
boosted trees, stopped early	0.217 and 0.225	0.777 and 0.776

On the volatility table, ridge, lasso and boosted stumps lie within 0.008 of each other. On the credit table, boosting and the forest tie, 0.029 above logistic regression.

The models of the course

model	a forecast is	settings to choose	reach for it when
linear regression	a weighted sum of the columns	none	the relationship is close to straight, and always as a baseline
ridge, lasso	the same sum, with the weights shrunk	<code>alpha</code>	there are many columns for the rows
logistic regression	a probability from a weighted sum	<code>C</code>	the target is a label and the weights should be read
k-nearest neighbours	the average of the closest rows	<code>n_neighbors</code>	there are few columns, all scaled
a tree	the mean of the box a row lands in	<code>max_depth,</code> <code>min_samples_leaf</code>	the rules themselves are wanted
a random forest	the average of many deep trees	<code>max_features,</code> <code>min_samples_leaf</code>	a strong model is needed with little tuning
boosted trees	the sum of many small trees	<code>learning_rate,</code> <code>n_estimators,</code> depth	there are many rows, interactions and missing values

All seven follow the same pattern in code: create, fit, predict, with the settings chosen on validation rows.

When to reach for boosting

Boosted trees first

Many rows, columns that bend or interact, missing values, a mix of types. On the credit table boosting and the forest reached an AUC of 0.777, logistic regression 0.748.

A linear model first

Few rows, a noisy target, a relationship close to a straight line. On the volatility table ridge scored 0.218, and boosting stopped early 0.217 and 0.225.

Fit a baseline and a linear model first. Keep a boosted model only if it wins on validation rows that suit the data.

The workflow, start to finish

1. Load the table and look at it: shapes, types, missing values.
2. Build the target and the features from the past only.
3. Split by date, or at random when the rows are separate cases.
4. Score a baseline: the average, or the majority class.
5. Fit, and choose the settings on folds or on a later block.
6. Open the test rows once, for the model you chose.
7. Read the model: its coefficients, or its importance on the test rows.
8. Report the score beside the baseline's.

What you can now do

SESSIONS 1 TO 3

Work with data

Load, clean, reshape and plot a table with pandas and matplotlib, using loops, functions and dictionaries.

SESSION 4

Frame a prediction

Build features and a target from the past only, split by date, and pick a metric and a baseline first.

SESSIONS 5 AND 6

Fit and choose

Fit linear models, ridge and lasso, choose their settings on folds that suit the rows, and open the test rows once.

SESSIONS 8 AND 9

Classify

Logistic regression and k-nearest neighbours; AUC, precision and recall, thresholds set from costs, and calibration.

SESSIONS 11 AND 12

Grow trees

Trees, forests and boosted trees, tuned on the right rows, timed, and read through their importance.

SESSIONS 7 AND 10

Run the whole workflow

From a raw table to a scored forecast, on house prices in Aarhus and on power prices in West Denmark.

What stays with you

Averaging cannot remove bias

100 averaged stumps err 0.577 on the training days, against 0.600 for one. 100 stumps added one after another err 0.457.

More trees can overfit

At $\nu = 0.1$ the test error is lowest near 144 stumps, 0.221, and 0.265 at 3,000. Deeper trees turn up sooner.

XGBoost and LightGBM

Bins, a penalty on each tree, growth leaf by leaf: on the credit table the same trees fit about 25 times faster than in [GradientBoosting](#).

Importance, two ways

Counts of cuts ranked a column of random numbers first. Shuffles on the test rows ranked it last, at zero.

Boosting fits what is left

$\hat{f}_m = \hat{f}_{m-1} + \nu h_m$: each small tree h_m is fitted to the residuals, the negative gradient of the loss.

Stop on a later block

XGBoost stopped on 2022 at 244 stumps: 0.225. Stopped on the test days, it reported 0.204, a score those days chose.

Boosting a label

Each tree is fitted to $y_i - \hat{p}_i$ on the log-odds scale. On the credit table, 0.777: level with the forest.

Correlated columns share importance

Shuffled together, [late_now](#) and [months_late](#) cost 0.206 of AUC; a refit without either one loses under 0.02.

Where this goes next

before

Python and pandas; linear regression, ridge and lasso; logistic regression and k-nearest neighbours; trees and forests

Sessions 1 to 11

now

boosting, XGBoost and LightGBM, and reading a model

trees, added

next

a final case lecture, and the wrap-up of the course

to finish

Every model came with the same routine: a baseline to beat, settings chosen on rows that suit the data, and a test set opened once.

After this session **CASE**

- **Exercises:** the six days boosted by hand, the gradient behind the residuals, the number of trees chosen on 2022 and on the folds, XGBoost and LightGBM, importance measured four ways, and a target with nothing in it.
- **Case Part 12**, the last part: boosting for the risk report's forecast and warning, then every model fitted since Part 4 side by side, and the report as it would go out.

Stuck? Ask a friend, ask an AI for a hint rather than an answer, or email jobo@econ.au.dk.